

Multimodal Retrieval- Augmented Generation (mRAG) on MMDocIR

By: Kiran Vågen, Gabriel Paulo Maglalang Israel, Oluwatobi Togun, Ørjan Rode Østlyngen

AI disclosure

AI was used, so we enforced strict consistency. Key guidelines:

- Single source of truth in [config.py](#) and inheritance from base classes
- Domain based structure
- Keep code shallow (max ~5 indents)
- Linting formatting, docstrings
- Cache all steps for speed, with multiprocessing, threading, and logging

Tools & Libraries

- **Pdf Processor:** Docling
- **Vector Database:** Qdrant
- **Embedder:** Jina Clip 2 (Text + Images)
- **Exact Word Matching:** Bm25
- **Agent Framework:** LangChain
- **Agent ProcessFlow:** LangGraph

Preprocessing

- Unstructured (bad for semantic chunking)
- Docling
- Key problem: capping chunk sizes so the embedder doesn't crash

Phase	Duration (seconds)
PDF Extraction (docling)	3898.3251
PDF Chunking	6.0412
Total Pipeline Runtime	3904.3663

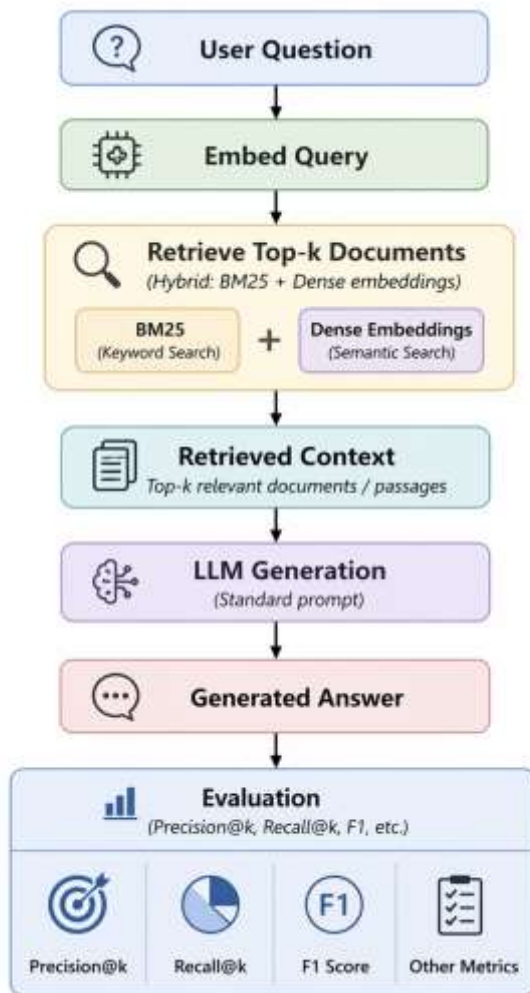
Building multimodal index: sliding_window	1526.6412
Building multimodal index: semantic	1457.9792
Building multimodal index: hierarchical	1511.678
Building multimodal index: enhanced_hierarchical	1640.878
Total Pipeline Runtime	6137.1765

preprocessed
{ } all_documents.json
{ } chunks_enhanced_hierarchical....
{ } chunks_fixed_size.json
{ } chunks_hierarchical.json
{ } chunks_semantic.json
{ } chunks_sliding_window.json

```
22 def preprocessing_pipeline():
23     """Runs the preprocessing pipeline."""
24     config = BaselineConfig()
25     tracker = MetricsTracker(logger)
26     log_and_time = tracker.log_and_time
27
28     with log_and_time('Total Pipeline Runtime'):
29
30         with log_and_time('Extracting PDFs using docling'):
31             all_documents = process_all_pdfs(config.PDFS_DIR)
32
33         if not all_documents:
34             raise ValueError("PDF extraction returned no documents.")
35
36         with log_and_time(f'Saving extracted JSON to {config.PREPROCESSED_DOCS_DIR}'):
37             save_read_pdf_data(all_documents, path=config.PREPROCESSED_DOCS_DIR)
38
39         with log_and_time('Loading documents for chunking'):
40             all_documents = load_read_documents(config.PREPROCESSED_DOCS_DIR)
41
42         with log_and_time('Running chunking methods and saving outputs'):
43             chunk_result = chunk_and_save_pdf_data(
44                 all_documents, output_dir=config.PREPROCESSED_DATA_DIR
45             )
46
47     tracker.print_timing_summary()
48
49     tracker.save_to_csv(tracker.timing_data, config.PREPROCESSING_TIME_CSV)
50
51     return {
52         "status": "success",
53         "processed_count": len(all_documents),
54         "chunk_info": chunk_result,
55         "timing": tracker.timing_data
56     }
57
58 if __name__ == "__main__":
59     result = preprocessing_pipeline()
60     print(f"Preprocessing pipeline completed successfully: {result}")
61
62     # how to run:
63     # DAT560project/src> python -m pipelines.preprocessing_pipeline
```

System 1. Baseline

All pipeline process flow run through a single function.



```
def run_single_experiment(
    experiment_name: str,
    config,
    pipeline_class,
    test_data: List[Dict[str, Any]],
    force_rebuild: bool = False,
    run_single_query: bool = False,
    results_csv_path: Optional[Path] = RESULTS_CSV,
    time_csv_path: Optional[Path] = PIPELINE_TIME_CSV,
):
    """Run a single pipeline configuration and log metrics."""
    logger.info(f"\n{'*' * 80}STARTING EXPERIMENT: {experiment_name}\n")

    full_pipeline_name = f"{experiment_name} ({pipeline_class.__name__})"
    tracker = MetricsTracker(logger, pipeline_name=full_pipeline_name,
                             log_and_time = tracker.log_and_time)

    pipeline = pipeline_class(config)

    with log_and_time("Total Pipeline Runtime"):

        with log_and_time("Index Build/Load"):
            pipeline.build_index(force_rebuild=force_rebuild)

        with log_and_time("Component Initialization"):
            pipeline.initialize_components()

        if run_single_query and not test_data:
            raise ValueError("no test data found")

        with log_and_time("Single Query Test"):
            run_single_query_test(pipeline, test_data[0]['question'],

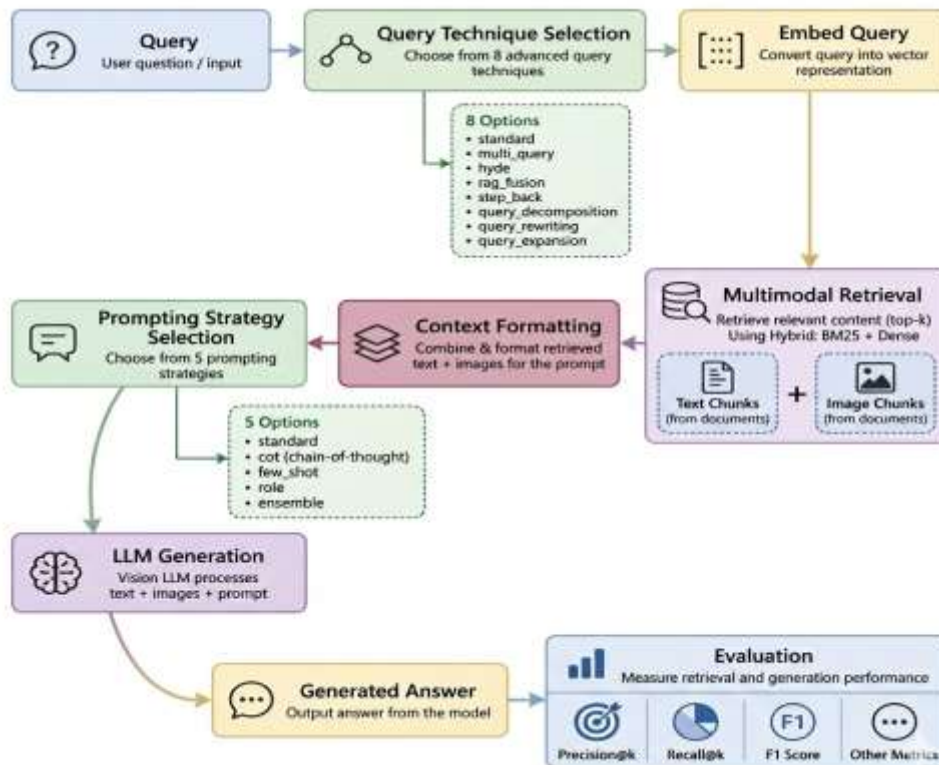
        # Run evaluation
        with log_and_time("Full Evaluation Time"):
            metrics = pipeline.evaluate(test_data, use_technique=True,

    tracker.save_to_csv(metrics, results_csv_path)
    tracker.save_to_csv(tracker.timing_data, time_csv_path)
    tracker.print_metrics(f"RESULTS FOR: {experiment_name}", metrics)

    return metrics

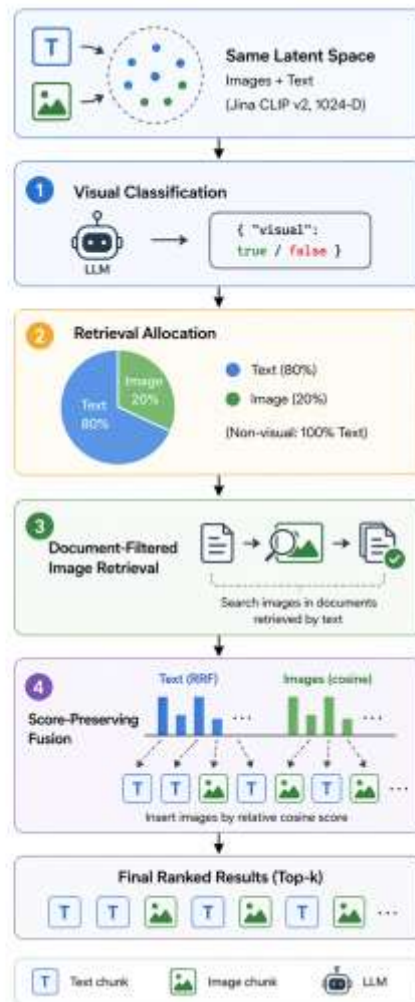
# How to use - DATS60project> python src/main.py:
# TO RUN BASELINE:
config = BaselineConfig()
pure_text_data = load_train_data(config.TEST_JSONL)
run_single_experiment(
    experiment_name="Baseline RAG",
    config=config,
    pipeline_class=BaselineRAGPipeline,
    test_data=pure_text_data[:config.EVAL_SUBSET_SIZE],
    force_rebuild=False,
    run_single_query=True
)
```

System 2. Advanced



System 2. Multimodal

- Same latent space
- Visual Classification
- Retrieval Allocation
- Document Filtered Image Retrieval
- Score Preserving Fusion



System 2. Ablations time (first run - not cached)

```
1 result = pd.read_csv("../results/pipeline_time.csv")
```

```
2 result
```

✓ 0.0s

Bottleneck

	Pipeline	model	Index Build/Load	Component Initialization	Single Query Test	Full Evaluation Time	Total Pipeline Runtime
0	1_Baseline (BaselineRAGPipeline)	qwen3-vl:8b-instruct	45.8012	0.0158	0.9621	77.4237	124.2028
1	2_Chunk_Ablation_sliding_window (AdvancedRAGPi...	qwen3-vl:8b-instruct	43.5951	0.0135	1.4864	241.9458	287.0409
2	2_Chunk_Ablation_semantic (AdvancedRAGPipeline)	qwen3-vl:8b-instruct	52.1082	0.0158	1.3504	216.6868	270.1612
3	2_Chunk_Ablation_hierarchical (AdvancedRAGPipe...	qwen3-vl:8b-instruct	44.5680	0.0295	0.6080	111.0101	156.2155
4	2_Chunk_Ablation_enhanced_hierarchical (Advanc...	qwen3-vl:8b-instruct	57.4848	0.0197	1.9327	238.4471	297.8843
5	3_Query_Ablation_multi_query (AdvancedRAGPipe...	qwen3-vl:8b-instruct	50.5924	0.0177	1.7943	287.2127	339.6172
6	3_Query_Ablation_rag_fusion (AdvancedRAGPipeline)	qwen3-vl:8b-instruct	50.4469	0.0212	1.8060	302.3411	354.6152
7	3_Query_Ablation_step_back (AdvancedRAGPipeline)	qwen3-vl:8b-instruct	52.1548	0.0209	0.9022	190.0842	243.1620
8	3_Query_Ablation_hyde (AdvancedRAGPipeline)	qwen3-vl:8b-instruct	50.1457	0.0191	0.7408	131.9696	182.8752
9	3_Query_Ablation_query_decomposition (Advanced...	qwen3-vl:8b-instruct	51.3524	0.0153	1.6245	299.6719	352.6641
10	3_Query_Ablation_query_rewriting (AdvancedRAGP...	qwen3-vl:8b-instruct	49.5442	0.0194	0.9312	172.7806	223.2753
11	3_Query_Ablation_query_expansion (AdvancedRAGP...	qwen3-vl:8b-instruct	50.2205	0.0232	1.5789	258.3494	310.1721
12	4_Prompt_Ablation_few_shot (AdvancedRAGPipeline)	qwen3-vl:8b-instruct	51.5251	0.0167	0.6255	111.5888	163.7560
13	4_Prompt_Ablation_role (AdvancedRAGPipeline)	qwen3-vl:8b-instruct	58.3860	0.0272	0.8716	111.7207	171.0056
14	4_Prompt_Ablation_cot (AdvancedRAGPipeline)	qwen3-vl:8b-instruct	52.5619	0.0179	0.7344	113.4645	166.7798
15	5_Multimodal_True (AdvancedRAGPipeline)	qwen3-vl:8b-instruct	49.3055	0.0303	0.5386	117.1804	167.0548

System 2. Ablations Metrics (Initial results)

	Pipeline	Test Size	LLM Model	Embedding Model	precision@1	recall@1	exact_match	token_f1
0	1_Baseline (BaselineRAGPipeline)	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.52	0.52	0.12	0.1593
1	2_Chunk_Ablation_sliding_window (AdvancedRAGPi...	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.84	0.84	0.1333	0.1922
2	2_Chunk_Ablation_semantic (AdvancedRAGPipeline)	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.86	0.86	0.12	0.1719
3	2_Chunk_Ablation_hierarchical (AdvancedRAGPipe...	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.8733	0.8733	0.1133	0.1677
4	2_Chunk_Ablation_enhanced_hierarchical (Advanc...	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.86	0.86	0.0933	0.1479
5	3_Query_Ablation_multi_query (AdvancedRAGPipe...	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.8067	0.8067	0.1333	0.1992
6	3_Query_Ablation_rag_fusion (AdvancedRAGPipeline)	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.6933	0.6933	0.1333	0.1939
7	3_Query_Ablation_step_back (AdvancedRAGPipeline)	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.7133	0.7133	0.1267	0.1827
8	3_Query_Ablation_hyde (AdvancedRAGPipeline)	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.6467	0.6467	0.0733	0.1371
9	3_Query_Ablation_query_decomposition (Advanced...	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.7067	0.7067	0.1333	0.1882
10	3_Query_Ablation_query_rewriting (AdvancedRAGP...	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.84	0.84	0.12	0.1683
11	3_Query_Ablation_query_expansion (AdvancedRAGP...	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.7867	0.7867	0.12	0.1869
12	4_Prompt_Ablation_few_shot (AdvancedRAGPipeline)	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.8467	0.8467	0.0933	0.1515
13	4_Prompt_Ablation_role (AdvancedRAGPipeline)	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.8467	0.8467	0.08	0.1381
14	4_Prompt_Ablation_cot (AdvancedRAGPipeline)	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.8467	0.8467	0.1667	0.207
15	5_Multimodal_True (AdvancedRAGPipeline)	150	qwen3-vl:8b-instruct	jinaai/jina-clip-v2	0.82	0.82	0.1333	0.1739

We decided to keep using multimodal(image) as the default for system 2.

System 2. Combining strategies (Incremental optimization) on token_f1 score

	Pipeline	Test Size	LLM Model	Embedding Model	precision@5	recall@5	exact_match	token_f1
16								
17	Step1_ChunkOpt_feed_size (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.184	0.92	0.1267	0.1893
18	Step1_ChunkOpt_sliding_window (AdvancedRAGPipe...	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.1827	0.9133	0.1333	0.1922
19	Step1_ChunkOpt_semantic (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.1827	0.9133	0.12	0.1719
20	Step1_ChunkOpt_hierarchical (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.1827	0.9133	0.1133	0.1677
21	Step1_ChunkOpt_enhanced_hierarchical (Advanced...	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.1827	0.9133	0.0933	0.1479
22	Step2_QueryOpt_standard (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.1827	0.9133	0.1333	0.1922
23	Step2_QueryOpt_multi_query (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.1787	0.8933	0.14	0.1965
24	Step2_QueryOpt_rag_fusion (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.176	0.88	0.1467	0.2016
25	Step2_QueryOpt_step_back (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.176	0.88	0.1267	0.1741
26	Step2_QueryOpt_hyde (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.148	0.74	0.08	0.1354
27	Step2_QueryOpt_query_decomposition (AdvancedRA...	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.1747	0.8733	0.1267	0.1857
28	Step2_QueryOpt_query_rewriting (AdvancedRAGPip...	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.1827	0.9133	0.1467	0.2004
29	Step2_QueryOpt_query_expansion (AdvancedRAGPip...	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.1707	0.8533	0.1	0.1627
30	Step3_PromptOpt_standard (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.176	0.88	0.1467	0.2016
31	Step3_PromptOpt_few_shot (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.176	0.88	0.1	0.1614
32	Step3_PromptOpt_role (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.176	0.88	0.1	0.153
33	Step3_PromptOpt_cot (AdvancedRAGPipeline)	150	qwen3-vt8b-instruct	jinaai/jina-clip-v2	0.176	0.88	0.1467	0.1878

Using iterative optimization, selecting the best variant of each component at every step and carrying it forward into the next stage.

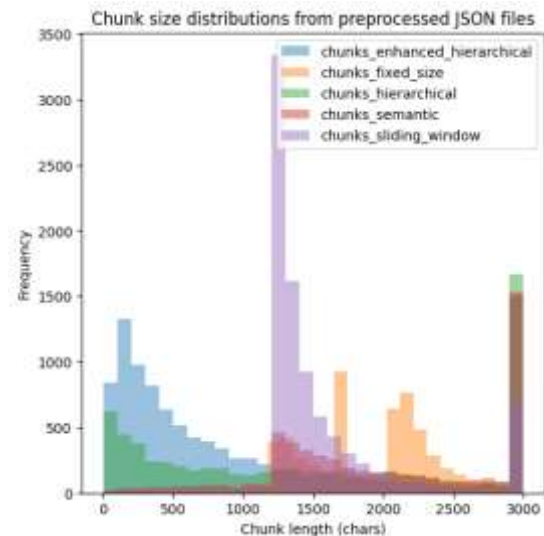
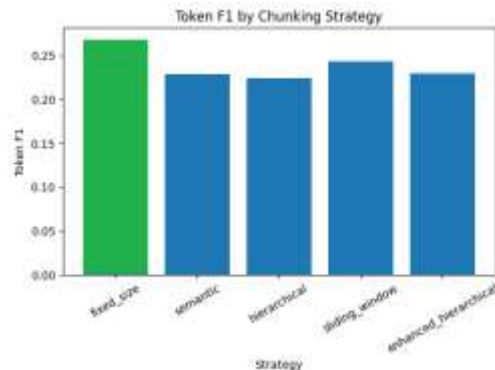
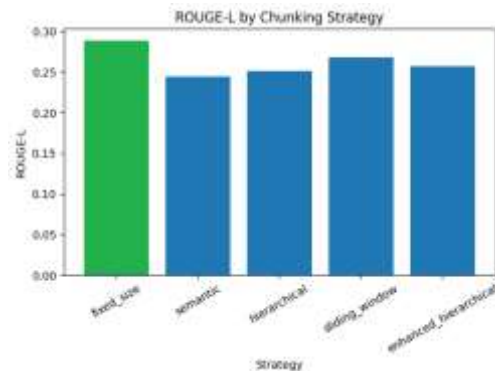
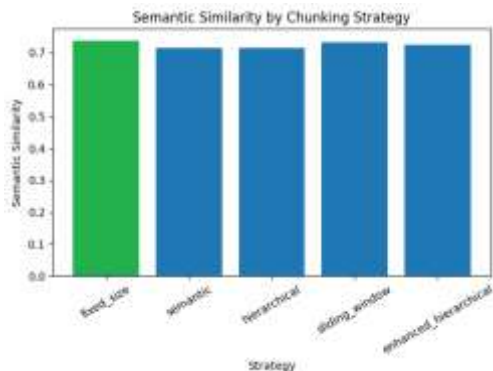
Step 1. Chunking_strategy: Sliding window

Step 2. Query technique: rag_fusion

Step 3. prompting strategy: standard

token_f1 score didnt do that well with this method

System 2. Chunking Strategies Generation Metrics



Best Generation Metric: Fixed-Size Chunking

Reason: Large enough chunk length to include all necessary information than other strategies

System 2. Chunking Strategies Retrieval Metrics

Chunking strategy	P@1	P@3	P@5	R@1	R@3	R@5	NDCG @1	MAP	MRR
Fixed-size	0.8467	0.300	0.1827	0.8467	0.900	0.9133	0.8467	0.8733	0.8733
Semantic	0.8667	0.2978	0.1800	0.8667	0.8933	0.9000	0.8667	0.8813	0.8813
Hierarchical	0.8667	0.3044	0.1027	0.8667	0.9133	0.9133	0.8667	0.8878	0.8878
Sliding Window	0.833	0.3022	0.1827	0.8333	0.9067	0.9133	0.8333	0.8680	0.8680
Enhanced Hierarchical	0.7133	0.2933	0.1800	0.7133	0.8800	0.9000	0.7133	0.7983	0.7983

System 2. Query Processing

-  Standard 
- Multi-Query
- RAG-Fusion
- Step-Back
- HyDe
- Query-Expansion
- Query-Rewriting
- Query-Decomposition

Query Technique	Precision @1	Recall @1	Token F1	Exact Match	Semantic Similarity
Standard	0.847	0.847	0.268	0.200	0.738
Step-Back	0.707	0.707	0.253	0.187	0.723
Multi	0.793	0.793	0.251	0.187	0.724
Rag-Fusion	0.687	0.687	0.230	0.173	0.698
HyDe	0.653	0.653	0.190	0.133	0.700
Expansion	0.780	0.780	0.227	0.147	0.700
Rewriting	0.833	0.833	0.227	0.173	0.700
Decomposition	0.700	0.700	0.244	0.167	0.714

System 2. Query Processing

- Model: 8b
- Worse Query
- Lose Semantic context
- HyDe Suffer Greatly
- Ensemble not good for RAG

System 2. Prompt Strategy

- Standard
- 🏰Chain Of Thought (CoT)🏰
- Few-Shot
- Role (Rag specialist)

Query Technique	Token F1	BLEU	ROUGE1	Exact Match	Semantic Similarity
Standard	0.2681	0.2325	0.2913	0.200	0.7377
CoT	0.2858	0.2492	0.3062	0.2267	0.6988
Few-Shot	0.1752	0.1330	0.1830	0.1133	0.6051
Role	0.1342	0.0915	0.1459	0.1000	0.5917

Why:

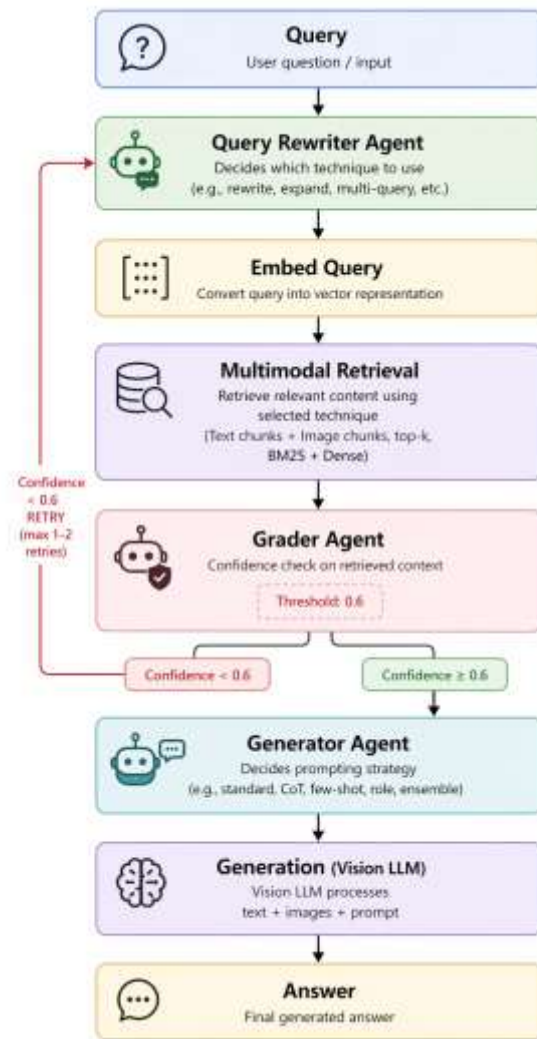
- Reasoning Questions
- Model: 8b
- Bad examples
- No good role for all

System 2. Combining strategies (Intuitive combination)

Chunk	Prompt	Query	Precision@1	Recall@1	Exact Match	Token F1
Enhanced Hier	CoT	Rag-Fusion	0.6467	0.6470	0.2130	0.2670
Semantic	CoT	Rag-Fusion	0.6467	0.6467	0.2130	0.2630
Semantic	CoT	HyDe	0.6733	0.6730	0.1800	0.2282
Fixed-size	CoT	Rag-Fusion	0.6467	0.6467	0.2067	0.2599
Fixed-Size	CoT	HyDe	0.6733	0.6733	0.1800	0.2187
Fixed-Size	CoT	Standard	0.8467	0.8467	0.2267	0.2858

System 3. Agentic mRAG

- Three Agents:
 - **Query Rewriter**
 - **Grader**
 - **Generator**
- LLM-based decision making
- **LangChain, LangGraph (StateGraph)**
 - For control flow, state management and orchestration
- Confidence-based retry-mechanism
- Structured Agent Outputs (**Pydantic** models/schemas)
 - Capture reasoning, confidence and choices/decisions
- Complete Decision Audit Trail
 - State accumulates all agent reasoning for potential auditing purposes



System 3. Agentic mRAG

- **Retrieval dominance:** System 2 retrieves the correct document in 91.3% of cases (top-5) vs. System 3's 66.7% — **System 3 fails entirely 1 in 3 times**; maybe root cause of downstream failures
- **Document ranking quality:** System 2's MAP/MRR of 0.873 vs. System 3's 0.598 means System 2 surfaces the correct document ~50% faster (avg rank 1.15 vs. 1.67).
- **Generation quality gap:** System 2 achieves **double the lexical overlap** (Token F1 0.286 vs 0.145) and produces perfect matches **twice as often** (22.7% vs 10.7%).
- **Cascading errors:** Bad query rewriting → poor retrieval → garbage context for generation. System 2's *fixed* pipeline avoids this error cascade entirely.

EVALUATION SUMMARY

No. of questions evaluated: 150

Total time: 2347.11s

System 3

Retrieval Metrics:

```
precision@1: 0.5466666666666666
precision@3: 0.21999999999999995
precision@5: 0.13333333333333333
recall@1: 0.5466666666666666
recall@3: 0.66
recall@5: 0.6666666666666666
page_recall@1: 0.0
page_recall@3: 0.0
page_recall@5: 0.0
ndcg@1: 0.5466666666666666
ndcg@3: 0.6129348485952403
ndcg@5: 0.6158060256490628
map: 0.5983333333333333
mrr: 0.5983333333333333
```

Generation Metrics:

```
token_f1: 0.1450515425354135
bleu: 0.12118827418960992
rouge1: 0.1689223889683923
rouge2: 0.07385185185185185
rougeL: 0.16350736920554643
exact_match: 0.10666666666666667
contains_match: 0.13333333333333333
semantic_similarity: 0.6311830016970634
```

System 2

Retrieval Metrics:

```
precision@1: 0.8467
precision@3: 0.3
precision@5: 0.1827
recall@1: 0.8467
recall@3: 0.9
recall@5: 0.9133
page_recall@1: 0.0
page_recall@3: 0.0
page_recall@5: 0.0
ndcg@1: 0.8467
ndcg@3: 0.8777
ndcg@5: 0.8834
map: 0.8733
mrr: 0.8733
```

Generation Metrics:

```
token_f1: 0.2858
bleu: 0.2492
rouge1: 0.3062
rouge2: 0.1371
rougeL: 0.2999
exact_match: 0.2267
contains_match: 0.2667
semantic_similarity: 0.6988
```

Final conclusion

System	Chunk	Prompt	Query	Precision@1	Recall@1	Exact Match	Token F1
1	Fixed-Size	Standard	Standard	0.5200	0.5200	0.1200	0.1593
2	Fixed-Size	CoT	Standard	0.8467	0.8467	0.2267	0.2858
3	Semantic	LLM	LLM	0.5466	0.5466	0.1067	0.1451

General adoptions and lessons learnt

Programming:

- Single source of truth through config and inheritance solves a lot of bug issues
- Caching all results incrementally is key for speed and motivation
- Going back to redefine preprocessing multiple times is very important
- Docling > Unstructured > pdfplumber for pdf preprocessing
- A combination of intuition rather than a programmatic approach can be better when deciding on combination of strategies

What is missing / Improvements

Given more time we would have:

- Followed the Strategy pattern for improved modularity, using a single mRAG class that accepts initialized components as inputs.
- Considered combined the strategies in all possible combinations to optimize a weighting of different metrics (precision, f1_score, ..)